

Q86. What is JDBC? Why is it used?

Simple Answer

JDBC (Java Database Connectivity) is a standard Java API used to connect Java applications with relational databases.

It allows applications to execute SQL queries, retrieve data, update records, and interact with databases in a database-independent manner.

Detailed Explanation

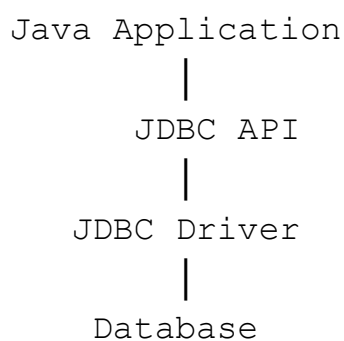
Using JDBC, a Java application can:

- Connect to a database
- Execute SQL statements
- Retrieve query results
- Insert, update, and delete records
- Manage database transactions

JDBC works with databases such as:

- MySQL
- Oracle
- PostgreSQL
- SQL Server

JDBC Architecture



Real-Time Example

An online shopping application uses JDBC to:

- Validate user login
- Fetch product details
- Save customer orders

- Update inventory

Code Example

```
Connection connection =  
    DriverManager.getConnection(  
        url, username, password);
```

Interview Tip

One of the most common interview questions:

What is JDBC?

Always mention:

- Java API
- Database Connectivity
- SQL Execution

Key Points

- ✓ Standard Java API.
- ✓ Connects Java with databases.
- ✓ Executes SQL.
- ✓ Database independent.

Q87. What are the steps to connect a Java application to a database using JDBC?

Simple Answer

The basic steps to connect to a database using JDBC are:

1. Load the JDBC Driver (optional in modern JDBC drivers).
2. Establish a database connection.
3. Create a Statement or PreparedStatement.
4. Execute the SQL query.
5. Process the ResultSet.
6. Close the resources.

Detailed Explanation

Step 1

Load Driver

```
Class.forName(...)
```

(Modern JDBC drivers often register automatically, so this step is usually not required.)

Step 2

Create Connection

```
DriverManager.getConnection(...)
```

Step 3

Create Statement

```
PreparedStatement
```

Step 4

Execute Query

```
executeQuery()
```

```
executeUpdate()
```

Step 5

Process Result

```
ResultSet
```

Step 6

Close Resources

Prefer **Try-with-Resources**.

Real-Time Example

Employee Management System

- Connect to MySQL.
- Execute:

```
SELECT * FROM employee;
```

- Display employee records.
- Close the connection.

Code Example

```
Connection con = DriverManager.getConnection(url, user,  
password);
```

```
PreparedStatement ps =  
    con.prepareStatement("SELECT * FROM employee");
```

```
ResultSet rs = ps.executeQuery();
```

Interview Tip

Interviewers often ask:

Which step is the most important?

Answer:

Always **close resources** properly to avoid connection leaks.

Key Points

- ✓ Connect.
- ✓ Create Statement.
- ✓ Execute SQL.
- ✓ Process ResultSet.

✓ Close resources.

Q88. What is the difference between Statement and PreparedStatement?

Simple Answer

Both are used to execute SQL queries.

- **Statement** executes static SQL queries.
- **PreparedStatement** executes precompiled parameterized SQL queries.

PreparedStatement is preferred because it improves performance and helps prevent SQL Injection.

Detailed Explanation

Statement	PreparedStatement
Static SQL	Parameterized SQL
Slower	Faster for repeated execution
Vulnerable to SQL Injection	Protects against SQL Injection
Query compiled every time	Query precompiled

Real-Time Example

Login Page

Incorrect approach:

```
SELECT * FROM users
WHERE username='Murali'
```

Using PreparedStatement:

```
SELECT * FROM users
WHERE username=?
```

This prevents malicious input from altering the SQL query.

Code Example

```
PreparedStatement ps =  
    connection.prepareStatement(  
        "SELECT * FROM employee WHERE id=?");  
  
ps.setInt(1,101);  
  
ResultSet rs = ps.executeQuery();
```

Interview Tip

One of the **Top JDBC interview questions**.

Whenever asked:

Which one should we use?

Answer:

PreparedStatement

because it is secure and more efficient.

Key Points

- ✓ Prevents SQL Injection.
- ✓ Better performance.
- ✓ Supports parameters.
- ✓ Preferred in enterprise applications.

Q89. What is ResultSet?

Simple Answer

A **ResultSet** is an object that stores the data returned by a SQL query.

It allows Java applications to read database records row by row.

Detailed Explanation

When `executeQuery()` is executed, it returns a `ResultSet`.

The cursor initially points **before the first row**.

Calling:

```
next()
```

moves the cursor to the next row.

Real-Time Example

Suppose the Employee table contains:

```
101 Murali
```

```
102 John
```

Using `ResultSet`, Java can iterate through each employee record and display it on the screen.

Code Example

```
ResultSet rs = ps.executeQuery();  
while(rs.next()){  
    System.out.println(  
        rs.getInt("id") +  
        " " +  
        rs.getString("name"));  
}
```

Interview Tip

Common interview question:

Where is the `ResultSet` cursor initially positioned?

Answer:

Before the first row.

Key Points

- ✓ Stores query results.
- ✓ Iterates row by row.
- ✓ Uses `next ()` to move the cursor.
- ✓ Retrieved from `executeQuery ()`.

✓ JDBC Topic Completed

Final JDBC Coverage

- 86. What is JDBC?
- 87. Steps to connect Java with Database
- 88. Statement vs PreparedStatement
- 89. What is ResultSet?

Final Review

I intentionally **did not include** separate questions on:

- `CallableStatement`
- Batch Processing
- Connection Pooling
- Transactions

These are valuable topics but are more commonly asked in **3–5 years** interviews or covered in Spring Data/JPA discussions.

The four selected questions represent the JDBC fundamentals that every **0–2 year** Java developer should know and are sufficient for your 100-question guide.

Current Progress

Topic	Questions
OOP	20
Collections	15
Core Java Basics	15
Exception Handling	8
Strings	6

Multithreading	8
Java 8 Features	10
Object Class	3
JDBC	4
Total Completed	89 Questions

Remaining (11 Questions)

This is where we'll make your guide stand out from a typical Core Java interview guide:

- **Generics** – 2 Questions (90–91)
- **Reflection** – 2 Questions (92–93)
- **IO/NIO** – 2 Questions (94–95)
- **Design Patterns** – 5 Questions (96–100)

These final 11 questions will replace lower-value or redundant topics and bring the guide up to date with what interviewers currently ask in Java interviews.

nice remaining response

Excellent! I think we have built a very strong guide so far. Now let's complete the **final 11 questions**. These are the additions that make your guide more relevant to modern Java interviews while still staying within the **100-question** limit.

Core Java Interview Guide

Final Section (Questions 90–100)

Topic 10: Generics

Q90. What are Generics? Why do we use them?

Simple Answer

Generics allow classes, interfaces, and methods to work with different data types while providing **compile-time type safety**.

They eliminate explicit type casting and reduce `ClassCastException`.

Detailed Explanation

Before Generics:

```
List list = new ArrayList();  
  
list.add("Java");  
  
String s = (String) list.get(0);
```

After Generics:

```
List<String> list = new ArrayList<>();  
  
list.add("Java");  
  
String s = list.get(0);
```

Advantages:

- Type Safety
- No Explicit Casting
- Reusable Code
- Compile-Time Error Checking

Real-Time Example

Spring Data JPA:

```
List<Employee>
```

instead of

```
List
```

Code Example

```
List<String> technologies = new ArrayList<>();  
  
technologies.add("Java");  
  
System.out.println(technologies.get(0));
```

Interview Tip

One of the favorite interview questions:

Why were Generics introduced?

Answer:

To provide **type safety** and eliminate explicit casting.

Key Points

- ✓ Compile-time type checking
- ✓ No explicit casting
- ✓ Better code readability
- ✓ Improves reusability

Q91. What are Wildcards (<?>, <? extends T>, <? super T>)?

Simple Answer

Wildcards provide flexibility when working with Generics.

- <?> → Unknown type
- <? extends T> → Read-only (Upper Bound)
- <? super T> → Write-oriented (Lower Bound)

Detailed Explanation

Think of the **PECS Rule**:

Producer → **Extends**

Consumer → **Super**

Real-Time Example

Reading employee records

↓

Use

```
List<? extends Employee>
```

Adding employee objects

↓

Use

```
List<? super Employee>
```

Code Example

```
List<? extends Number> numbers;
```

Interview Tip

Remember:

PECS

Producer → Extends

Consumer → Super

Key Points

- ✓ Makes Generics flexible
- ✓ Upper Bound → Read
- ✓ Lower Bound → Write
- ✓ Frequently used in frameworks

Topic 11: Reflection

Q92. What is Reflection in Java?

Simple Answer

Reflection is a feature that allows Java programs to inspect and manipulate classes, methods, fields, and constructors at runtime.

Detailed Explanation

Reflection enables developers to:

- Discover class information
- Invoke methods dynamically
- Access private fields
- Create objects dynamically

Real-Time Example

Spring Boot uses Reflection to:

- Create Beans
- Perform Dependency Injection
- Execute annotations

Code Example

```
Class<?> clazz = Employee.class;  
  
System.out.println(clazz.getName());
```

Interview Tip

Interview Question:

Which framework heavily uses Reflection?

Answer:

Spring Framework

Key Points

- ✓ Runtime inspection
- ✓ Dynamic object creation
- ✓ Used by Spring
- ✓ Used by Hibernate

Q93. How does Spring Framework use Reflection?

Simple Answer

Spring uses Reflection to create objects, inject dependencies, invoke methods, and process annotations without developers manually creating objects.

Detailed Explanation

Spring scans classes annotated with:

- `@Component`
- `@Service`
- `@Repository`
- `@Controller`

It creates objects dynamically using Reflection and manages them in the IoC container.

Real-Time Example

```
@Service
class EmployeeService{

}
```

Spring automatically creates the object.

Code Example

```
Class<?> clazz = Class.forName("EmployeeService");
```

Interview Tip

One-line answer:

Spring's Dependency Injection is powered by Reflection.

Key Points

- ✓ Bean creation
- ✓ Dependency Injection

✓ Annotation Processing

✓ Dynamic method invocation

Topic 12: IO / NIO

Q94. What is the difference between Java IO and Java NIO?

Simple Answer

Java IO is stream-based, whereas Java NIO is buffer-based and supports non-blocking operations.

Detailed Explanation

Java IO	Java NIO
Stream-based	Buffer-based
Blocking	Non-blocking
Slower	Faster
Older API	Introduced in Java 1.4

Real-Time Example

File Upload System

Large file transfer

↓

NIO provides better performance.

Code Example

```
Path path = Paths.get("sample.txt");
```

Interview Tip

Remember:

IO → Stream

NIO → Buffer

Key Points

✓ IO → Traditional

✓ NIO → High Performance

✓ NIO → Non-blocking

Q95. What is the difference between `InputStream` and `Reader`?

Simple Answer

`InputStream` reads binary data, whereas `Reader` reads character data.

Detailed Explanation

Use `InputStream` for:

- Images
- Videos
- PDFs

Use `Reader` for:

- Text Files
- JSON
- XML

Real-Time Example

Employee.csv

↓

`Reader`

EmployeePhoto.jpg

↓

`InputStream`

Code Example

```
FileReader reader = new FileReader("employee.txt");
```

Interview Tip

One-line answer:

Binary → InputStream

Text → Reader

Key Points

- ✓ InputStream → Binary
- ✓ Reader → Character
- ✓ Most common interview question

Topic 13: Design Patterns

Q96. What is a Design Pattern?

Simple Answer

A Design Pattern is a reusable solution to a commonly occurring software design problem. It provides a proven approach to designing maintainable, scalable, and reusable applications.

Detailed Explanation

Design Patterns are not ready-made code but best practices for solving recurring design problems.

Categories:

- Creational
- Structural
- Behavioral

Real-Time Example

Spring Framework uses many Design Patterns internally.

Interview Tip

Interviewers often ask for examples rather than definitions.

Key Points

- ✓ Reusable solutions
- ✓ Improve maintainability
- ✓ Used in enterprise applications

Q97. What is the Singleton Design Pattern?

Simple Answer

The Singleton Pattern ensures that only one instance of a class exists throughout the application and provides a global access point to that instance.

Real-Time Example

A Logger or Configuration Manager should have only one instance shared across the application.

Code Example

```
class Singleton {  
  
    private static final Singleton INSTANCE = new  
Singleton();  
  
    private Singleton() {}  
  
    public static Singleton getInstance() {  
        return INSTANCE;  
    }  
}
```

Interview Tip

Spring Beans are Singleton by default.

Key Points

- ✓ Single instance
- ✓ Global access
- ✓ Used for shared resources

Q98. What is the Factory Design Pattern?

Simple Answer

The Factory Pattern creates objects without exposing the object creation logic to the client.

Real-Time Example

A payment application creates different payment objects (UPI, Credit Card, Net Banking) based on user choice without the client knowing the creation details.

Interview Tip

Spring uses Factory patterns extensively, such as `BeanFactory`.

Key Points

- ✓ Encapsulates object creation
- ✓ Promotes loose coupling
- ✓ Improves maintainability

Q99. What is the Strategy Design Pattern?

Simple Answer

The Strategy Pattern defines a family of algorithms, encapsulates each one, and allows them to be selected at runtime.

Real-Time Example

An e-commerce application supports multiple payment strategies:

- UPI
- Credit Card
- Debit Card
- Net Banking

The appropriate strategy is selected based on the user's choice.

Interview Tip

Many Spring applications use the Strategy Pattern to choose different business implementations.

Key Points

- ✓ Runtime behavior selection
- ✓ Loose coupling
- ✓ Easy to extend

Q100. Which Design Patterns are used in the Spring Framework?

Simple Answer

Spring Framework uses several Design Patterns internally to provide flexibility, scalability, and maintainability.

Common patterns include:

- Singleton
- Factory
- Proxy
- Strategy
- Template Method
- Observer
- Dependency Injection (IoC)

Detailed Explanation

Examples:

- **Singleton** → Default scope of Spring Beans.
- **Factory** → `BeanFactory` and `ApplicationContext`.
- **Proxy** → Spring AOP.
- **Strategy** → Transaction management, validation.

- **Template Method** → `JdbcTemplate`, `RestTemplate`.

Real-Time Example

When you annotate a class with:

```
@Service
```

Spring creates and manages the bean using multiple design patterns behind the scenes.

Interview Tip

This is becoming one of the most frequently asked Spring-related Core Java questions because it connects Java fundamentals with enterprise development.

Key Points

- ✓ Spring heavily uses Design Patterns.
- ✓ Singleton is the default bean scope.
- ✓ Factory creates beans.
- ✓ Proxy powers AOP.
- ✓ Template Method simplifies database operations.